

Universidad de Guadalajara — CUTonalá

Ingeniería en Computación

Traductores de Lenguajes

Actividad 13

Generación de Código Ejecutable

Segura Valencia Francisco

Fecha: 07 de May de 2026

1. Objetivo

Implementar la fase final del compilador: la **generación de código ejecutable** a partir del Código de Tres Direcciones (TAC) optimizado. El generador traduce cada instrucción TAC a ensamblador simbólico, asigna direcciones de memoria a variables y temporales, produce una representación binaria del ejecutable y escribe archivos **.asm** y **.bin**.

2. Arquitectura del Compilador

Pipeline de 6 fases encadenadas. La fase resaltada (♦) es la implementada en esta actividad:

Fase	Clase / Archivo	Responsabilidad
Léxico	ScannerLexicoCpp (JFlex)	Tokenización
Sintáctico	parser (JCup)	Árbol sintáctico + acciones
Semántico	SymbolTable / SemanticError	Tipos, scopes, errores
TAC	TACGenerator	Código de tres direcciones
Optimización	TACOptimizer	4 pasos: dead code, CSE, copy, unused
Codegen	CodeGenerator ♦	ASM simbólico + binario

3. Implementación — CodeGenerator.java

1. Asignación de direcciones

asignarDirecciones() recorre el TAC y registra cada variable/temporal en un LinkedHashMap. Las variables reciben direcciones secuenciales desde DATA_BASE = 0x2000 con VAR_SIZE = 4 bytes cada una. Las etiquetas de salto (L1:, L2:) se detectan y excluyen del mapa de datos.

2. Traducción instrucción por instrucción

traducir() mapea cada línea TAC a 1–4 instrucciones ASM simbólicas. Maneja: etiquetas, marcadores begin/end, goto, ifFalse, if-goto (do-while), if!= - goto (switch), if==0-then (div-by-zero), return, read, llamadas con y sin resultado, y asignaciones (binarias, unarias, escalares).

3. Tabla de opcodes

buildOpcodes() devuelve un mapa String→int con 30 mnemónicos: LOAD(0x01), STORE(0x02), ADD(0x10), SUB(0x11), MUL(0x12), DIV(0x13), JMP(0x30), JEQ(0x31), JNE(0x32), CALL(0x40), RET(0x41), HALT(0xFF), etc.

4. Generación del binario

Formato personalizado de 18 bytes de cabecera: magic EXEC (4B) + versión (2B) + entry point (4B) + codeSize (4B) + dataSize (4B). Sección de código: cada instrucción real → 8 bytes (opcode<<24|addr, 0). Sección de datos: inicializada en cero por new byte[].

5. Salida de archivos

guardarArchivos() escribe .asm con el texto ensamblador completo (secciones .data y .text) y .bin con el ejecutable binario.

Constantes clave

Constante	Valor	Propósito
CODE_BASE	0x1000	Inicio de sección de código
DATA_BASE	0x2000	Inicio de sección de datos
INSTR_SIZE	4	Bytes por instrucción (addressing)
VAR_SIZE	4	Bytes por variable (int 32-bit)
Header size	18	magic(4)+version(2)+entry(4)+code(4)+data(4)

4. Caso Válido — valido_a13.cpp

Clase **Matematicas** con métodos *multiplicar* y *dividir*, más clase **Programa** con *main* que instancia, llama métodos y ejecuta un switch. Cero errores esperados.

✓ Sin errores — Compilación exitosa

TAC Original (extracto)

```
begin multiplicar
t1 = a * b
producto = t1
return producto
end multiplicar
begin dividir
if b == 0 then error "Division by zero"
t2 = a / b
cociente = t2
return cociente
end dividir
begin main
t3 = "Segura Valencia Francisco"
mat = create Matematicas
t4 = 6
x = t4
t5 = 3
y = t5
t6 = call mat.multiplicar, 2
prod = t6
t7 = call mat.dividir, 2
coc = t7
t8 = prod + coc
suma = t8
t9 = 1
op = t9
t10 = op
if t10 != 1 goto L2
```

```
t11 = 2
t12 = suma * t11
```

Optimizaciones aplicadas: 6

#	Tipo	Instrucción eliminada
1	Variable	temporal
2	Variable	no
3	Variable	no
4	Variable	temporal
5	Variable	temporal
6	Variable	temporal

Tabla de Símbolos — Asignación de Memoria

Código Ensamblador — extracto (primeras instrucciones)

```
; begin multiplicar
0x1000 LOAD R1, [0x2004] ; cargar 'a'
0x1004 LOAD R2, [0x2008] ; cargar 'b'
0x1008 MUL R1, R2 ; R1 ← R1 * R2
0x100C STORE [0x2000], R1 ; guardar en 't1'
0x1010 LOAD R1, [0x2000] ; cargar 't1'
0x1014 STORE [0x200C], R1 ; asignar a 'producto'
0x1018 LOAD R1, [0x200C] ; cargar valor de retorno 'producto'
0x101C RET ; retornar al llamador
; end multiplicar
; begin dividir
0x1020 LOAD R1, [0x2008] ; verificar divisor 'b'
0x1024 CMP R1, #0 ; comparar con cero
0x1028 JEQ _div_error ; saltar a error si divisor = 0
0x102C LOAD R1, [0x2004] ; cargar 'a'
0x1030 LOAD R2, [0x2008] ; cargar 'b'
0x1034 DIV R1, R2 ; R1 ← R1 / R2
0x1038 STORE [0x2010], R1 ; guardar en 't2'
0x103C LOAD R1, [0x2010] ; cargar 't2'
0x1040 STORE [0x2014], R1 ; asignar a 'cociente'
0x1044 LOAD R1, [0x2014] ; cargar valor de retorno 'cociente'
0x1048 RET ; retornar al llamador
; end dividir
; begin main
0x104C CALL _new_Matematicas ; crear instancia de Matematicas
0x1050 STORE [0x2018], R1 ; guardar referencia en 'mat'
0x1054 CALL _mat_multiplicar_n2 ; llamar mat.multiplicar, 2
0x1058 STORE [0x201C], R1 ; guardar retorno en 't6'
0x105C LOAD R1, [0x201C] ; cargar 't6'
0x1060 STORE [0x2020], R1 ; asignar a 'prod'
0x1064 CALL _mat_dividir_n2 ; llamar mat.dividir, 2
0x1068 STORE [0x2024], R1 ; guardar retorno en 't7'
0x106C LOAD R1, [0x2024] ; cargar 't7'
0x1070 STORE [0x2028], R1 ; asignar a 'coc'
0x1074 LOAD R1, [0x2020] ; cargar 'prod'
```

Cabecera del Ejecutable — hex dump (32 bytes)

```
0000: 45 58 45 43 01 00 00 00
0008: 10 00 00 00 02 48 00 00
0010: 00 58 01 00 10 00 00 00
0018: 00 00 01 00 10 04 00 00
```

5. Caso Inválido — invalido_a13.cpp

Archivo con errores semánticos y sintácticos deliberados: variable no declarada, tipo incompatible (bool+int), reasignación de constante, y más. El compilador debe reportar todos los errores y aun así generar TAC parcial.

X Se encontraron 4 error(es)

Errores Detectados

Tipo	Descripción
[Sintáctico	Falta ';' en declaración de variable
Semántico	Identificador 'sinDeclarar' no declarado
Semántico	Operador '+' requiere operandos numéricos (int/float/double), se encontraron tipos 'bool' y 'in
Semántico	Identificador 'noExiste' no declarado

TAC Parcial Generado (a pesar de errores)

```
begin dividir
if b == 0 then error "Division by zero"
t1 = a / b
resultado = t1
return resultado
end dividir
begin main
t2 = "Segura Valencia Francisco"
t3 = 5
t4 = sinDeclarar + t3
r = t4
t5 = true
flag = t5
t6 = 10
t7 = flag + t6
t8 = 50
LIMITE = t8
t9 = 100
LIMITE = t9
t10 = 20
y = t10
t11 = noExiste > y
ifFalse t11 goto L1
t12 = 0
y = t12
```

6. Video de Demostración

Video de Demostración

<https://youtu.be/placeholder-a13>

El video muestra la ejecución del compilador sobre ambos casos de prueba, con énfasis en la generación de código ASM y el binario producido.

7. Conclusiones

- Se implementó CodeGenerator.java con traducción completa de TAC → ASM simbólico.
- La asignación de memoria es automática: variables y temporales reciben direcciones en DATA_BASE (0x2000) de 4 bytes cada una.
- El generador cubre todos los patrones TAC del compilador: aritmética, comparaciones, saltos condicionales/incondicionales, llamadas a métodos, creación de objetos, return y read.
- El ejecutable binario sigue un formato de cabecera propia (magic EXEC) con secciones de código (opcode<<24|addr) y datos.
- Los archivos .asm y .bin se generan automáticamente junto al .cpp de entrada.
- El pipeline completo (léxico→semántico→TAC→optimización→codegen) funciona en una sola ejecución del JAR.